Solution Requirements

Date	15 July 2026
Team ID	XXXX
Project Name	Emotion Detection and Learning Support Engine
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

- **Functional Requirements (FR)** define specific behaviors, functions, or features of the system (What the system should do).
- **Non-Functional Requirements (NFR)** specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should be).

Step 1: Functional Requirements (FR)

S. No	Requirement Category	Requirement Description	Priority
1	Authentication	The system does not require user login or identity verification. It is a single-session, open-access tool where a student directly enters text describing their learning challenge to receive support; no personal credentials are collected.	Low
2	Authorization Levels	Single user role in the current build (Student/Learner). No admin panel or differentiated access levels exist since there is no login system; all users interact with the same functionality.	Low
3	External Interfaces	Integrates with the Google Gemini API (generative AI) to generate personalized, empathetic learning-support responses based on the detected emotion, with an automatic template-based fallback if the API is unavailable. No other third-party database or payment API is used.	High
4	Transactions Processing	Core data flow: student text input -> TextPreprocessor cleaning (regex + NLTK tokenization) -> emotion classification via two independent models - a BiLSTM network (TensorFlow/Keras) and a fine-tuned DistilBERT transformer (PyTorch + HuggingFace Transformers) - each producing a keyword-boosted probability distribution -> the BiLSTM's top emotion label is passed as prompt context to the Gemini API -> a personalized learning-support response is returned and displayed in the Streamlit UI.	High
5	Reporting	The system generates an in-session analytics dashboard (emotion distribution, emotional journey over time, and breakdowns by study field/model) built with Plotly, scoped to the current browser session. Separately, every interaction is also permanently appended to emotion_response_examples.csv and emotion_response_mapping.csv on disk, persisting across sessions for future analysis or retraining - this CSV log is distinct from the in-session dashboard view.	Medium

S. No	Requirement Category	Requirement Description	Priority
6	Business Rules	When the system detects negative emotions (Frustrated, Confused, Bored) it prioritizes supportive, simplified guidance and an encouraging next step; when it detects positive emotions (Confident, Curious) it prioritizes advanced content or deeper material, per the field-specific response templates.	Medium
7	Compliance to Laws or Regulations	As the tool processes student-written text, it follows responsible-AI practices: no login credentials or persistent personal identifiers are collected; the .env file containing the Gemini API key is excluded from version control (.gitignore) to prevent credential leakage.	Medium
8	Other	The system uses prompt engineering (field name, detected emotion, and confidence score embedded in the Gemini prompt) to keep generated responses educational, field-specific, and encouraging in tone.	Low

Step 2: Non-Functional Requirements (NFR)

S. No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	The system should classify the emotion from input text (across both BiLSTM and BERT) and return a Gemini-generated support response quickly enough to feel conversational.	End-to-end response (dual-model inference + Gemini API call) in under 5 seconds
2	Scalability	If deployed via a platform such as Streamlit Community Cloud or Hugging Face Spaces, the app should comfortably handle typical classroom-scale usage without degradation. (Exact deployment platform to be finalized.)	Supports up to 50-100 concurrent user sessions on the chosen hosting platform
3	Security & Data Privacy	No login credentials are stored. The Gemini API key is kept in a local .env file excluded from version control. Interaction logs (text + detected emotion) are appended to local CSV files and are not linked to any identifiable user account.	All API calls over HTTPS; no login credentials collected; API keys never committed to source control
4	Reliability & Availability	Availability depends on the chosen hosting platform and the Gemini API; the app degrades gracefully (falling back to template responses) if the Gemini API is temporarily unavailable, rather than failing outright.	Template-fallback response shown within the same session if Gemini API call fails
5	Usability & Accessibility	The Streamlit interface is designed to be simple and intuitive for students with no technical background, with a clear field selector, text input, quick-example prompts, and readable emotion/response output.	Usable by a first-time student within 1 minute, no training required
6	Maintainability	The codebase is modularized into text preprocessing (src/preprocessing.py), BiLSTM inference (src/model.py), BERT inference (src/bert_model.py), keyword-based boosting (src/predict.py), and the Streamlit UI/orchestration layer (app.py) for easier updates and portability across OS (Windows/Linux/macOS).	Modular Python codebase; runs on Python 3.9+ with requirements.txt-based setup